CS535: Algorithm Design and Analysis

Trees

Prof. Rohit P. Singh

Department of Computer Science

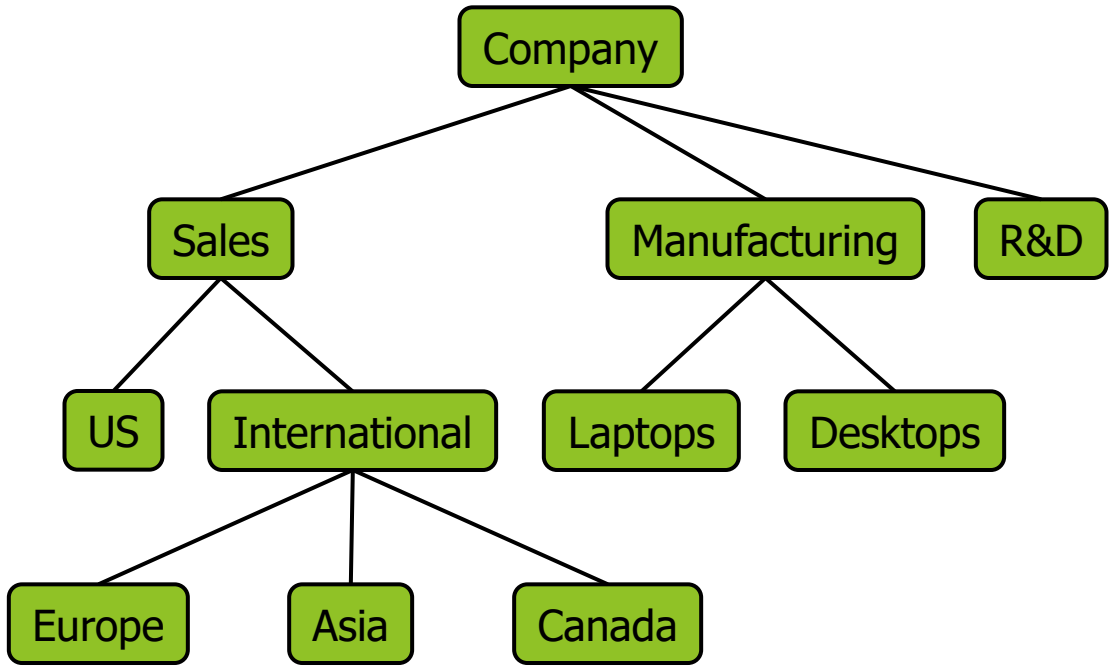
University of Wisconsin, Milwaukee

September 18, 2024

Readings: Chapter 2.3

What is a Tree

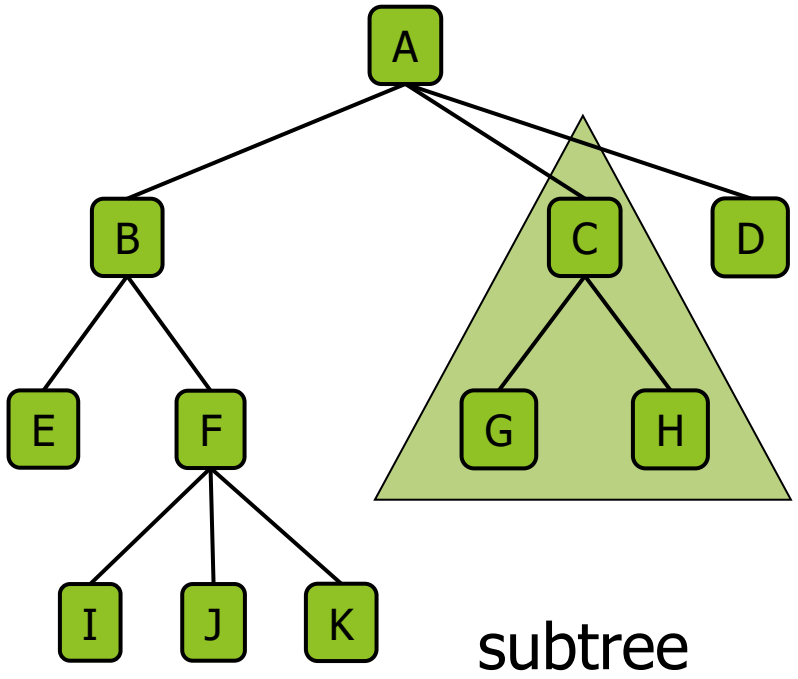
- In computer science, a tree is an abstract model of a hierarchical structure
- A tree consists of nodes with a parent-child relation
- Applications:
  - Organization charts
  - File systems
  - Programming environments



Tree Terminology

- **Root**: node without parent (A)
- **Internal node**: node with at least one child (A, B, C, F)
- **External node** (a.k.a. leaf): node without children (E, I, J, K, G, H, D)
- **Ancestors** of a node: parent, grandparent, grand-grandparent, etc.
- **Depth** of a node: number of ancestors
- **Height** of a tree: maximum depth of any node (3)
- **Descendant** of a node: child, grandchild, grand-grandchild, etc.

- **Subtree**: tree consisting of a node and its descendants



Tree ADT

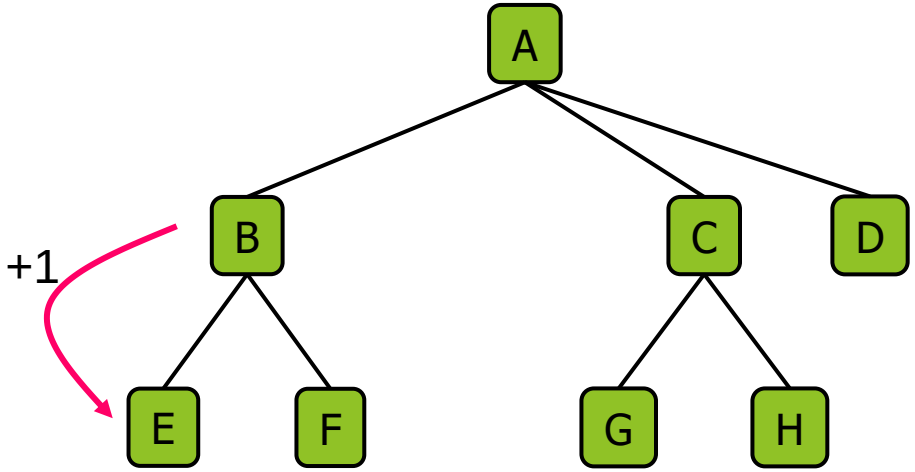
- We use *positions* to abstract nodes
- Generic methods:
  - integer size()
  - boolean isEmpty()
  - objectIterator elements()
  - positionIterator positions()
- Accessor methods:
  - position root()
  - position parent(p)
  - positionIterator children(p)

- Query methods:
  - boolean isInternal(p)
  - boolean isExternal(p)
  - boolean isRoot(p)
- Update methods:
  - swapElements(p, q)
  - object replaceElement(p, o)
- Additional update methods may be defined by data structures implementing the Tree ADT

Tree Depth

- If v is the root, then the depth of v is 0
- Otherwise, the depth of v is one plus the depth of the parent of v

```
Algorithm depth(T,v)
    If T.isRoot(v) then
        return 0
    else
        return
            1+depth(T,T.parent(v))
```



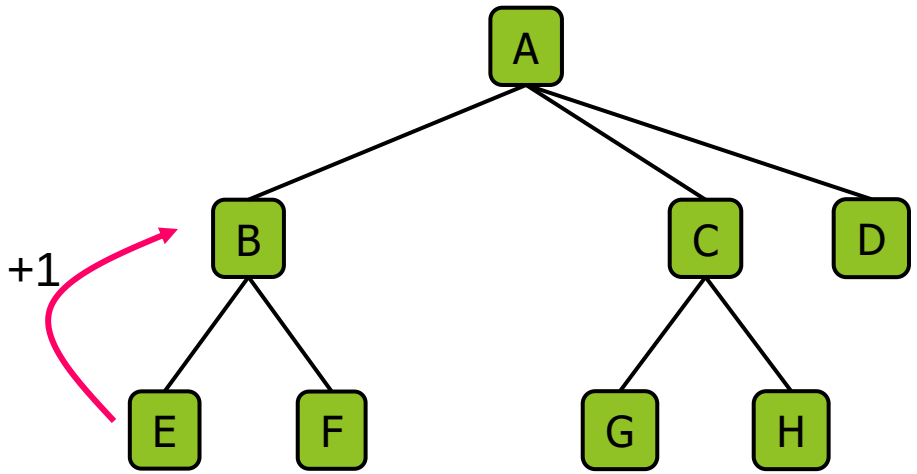
Running time: $O(d_v)$,

where dv is the depth of v

Tree (or Subtree) Height

- A “bad” algorithm: Check the depth of each node, then take the maximum.
- If v is an external node, then the height of v is 0
- Otherwise, the height of v is one plus the maximum height of the children of v
- The height of a tree T is the height of the root of T

```
Algorithm height(T,v)
    If T.isExternal(v) then
        return 0
    else
        h = 0
        for each child w of v do
            h=max(h,height(T,w)
        return 1+h
```



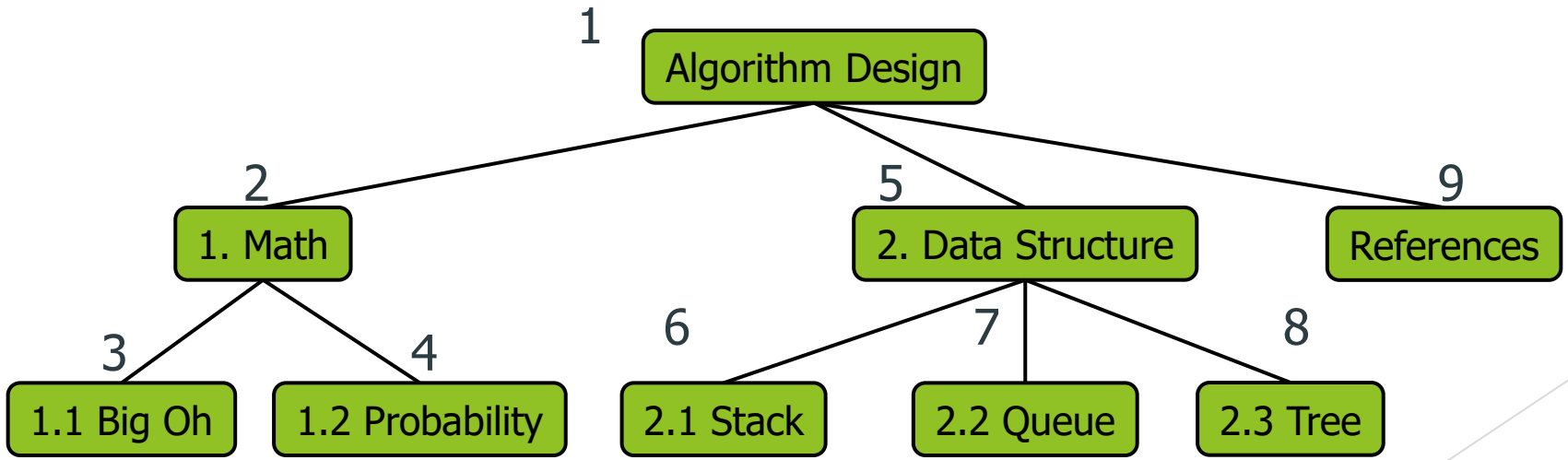
Running time: O(n),

where n is the number of nodes

Preorder Traversal

- A traversal visits the nodes of a tree in a systematic manner
- In a preorder traversal, a node is visited before its descendants
- Application: print a structured document (e.g., a textbook)

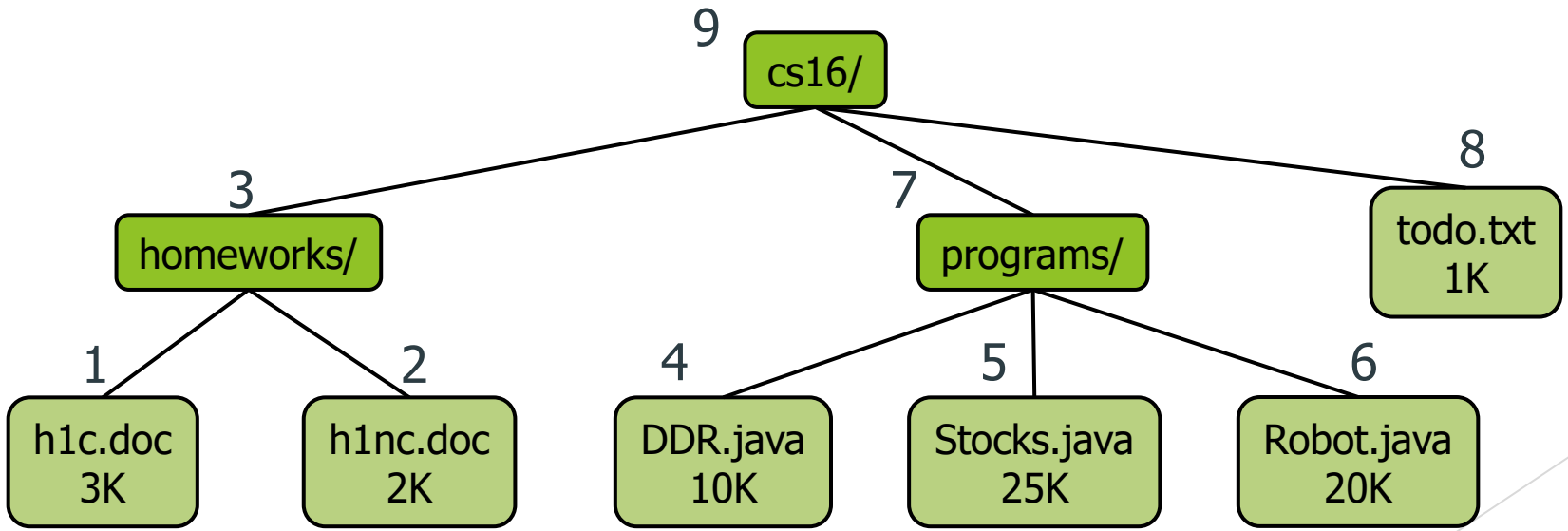
Algorithm ***preOrder***(***v***)
  visit(***v***)
  for each child ***w*** of ***v***
    preorder (***w***)



Postorder Traversal

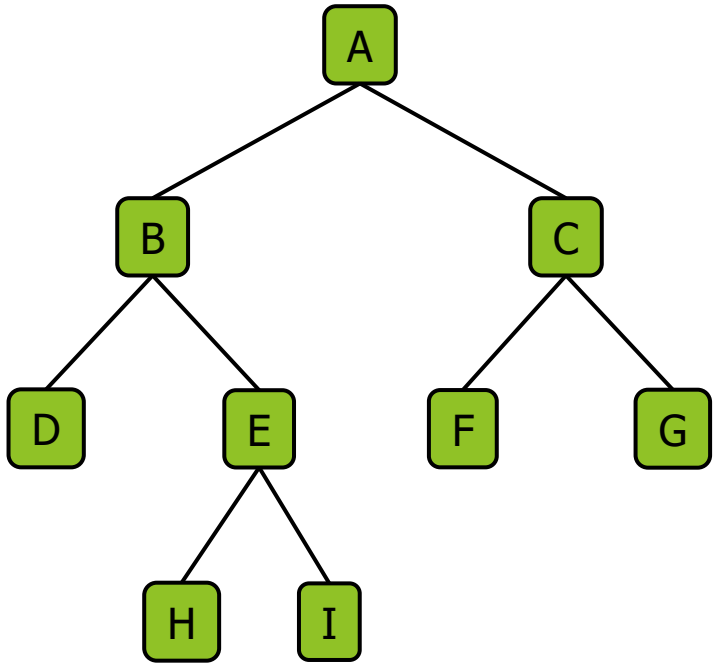
- In a postorder traversal, a node is visited after its descendants
- Application: compute space used by files in a directory and its subdirectories

Algorithm ***postOrder***(***v***)
  for each child ***w*** of ***v***
    postOrder (***w***)
  visit(***v***)



Binary Tree

- A binary tree is a tree with the following properties:
  - Each internal node has two children
  - The children of a node are an ordered pair
- We call the children of an internal node left child and right child
- Alternative *recursive definition*: a binary tree is either
  - a tree consisting of a single node, or
  - a tree whose root has an ordered pair of children, each of which is a binary tree



Binary Tree ADT

- The Binary Tree ADT extends the Tree ADT, i.e., it inherits all the methods of the Tree ADT
- Additional methods:
  - position leftChild(p)
  - position rightChild(p)
  - position sibling(p)
- Update methods may be defined by data structures implementing the BinaryTree ADT
  - swapElements(p, q)
  - object replaceElement(p, o)

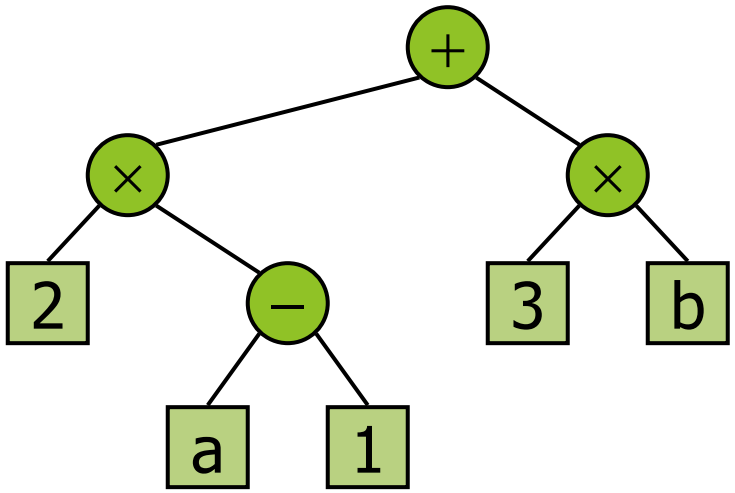
- Applications:
  - arithmetic expressions
  - decision processes
  - searching

Arithmetic Expression Tree

- Binary tree associated with an arithmetic expression
  - internal nodes: operators
  - external nodes: operands

- *Example*: arithmetic expression tree for the expression:

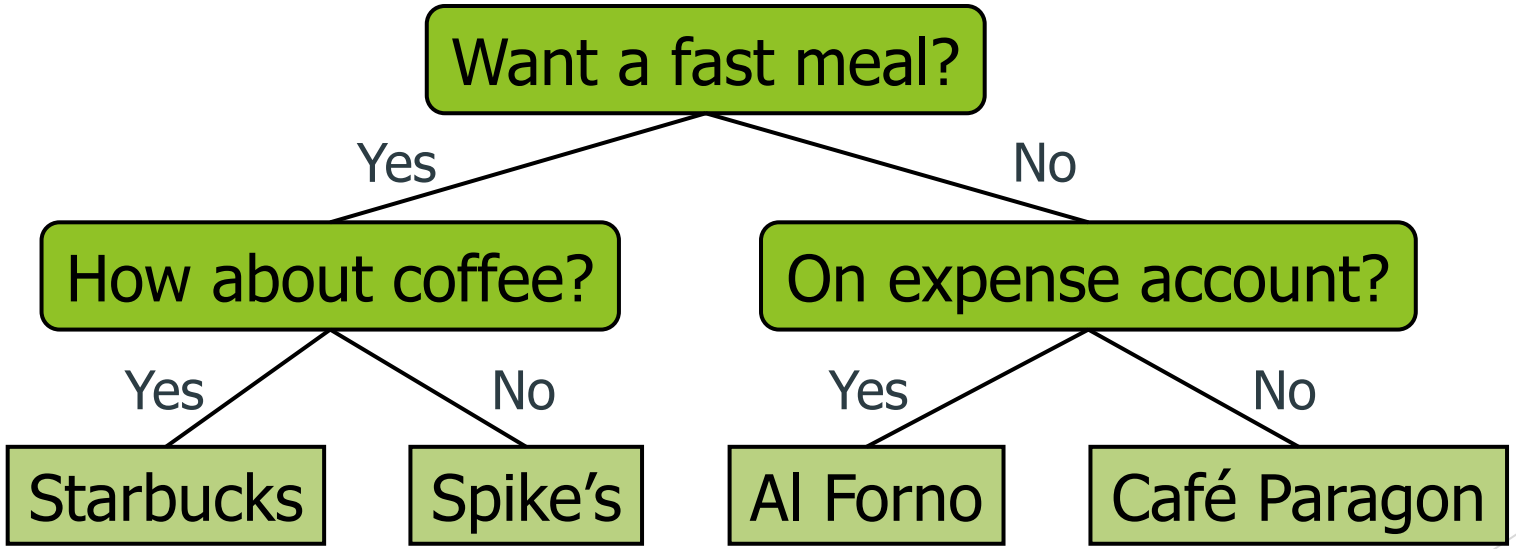
  $(2 \times (a - 1) + (3 \times b))$



Decision Tree

- Binary tree associated with a decision process
  - internal nodes: questions with yes/no answer
  - external nodes: decisions

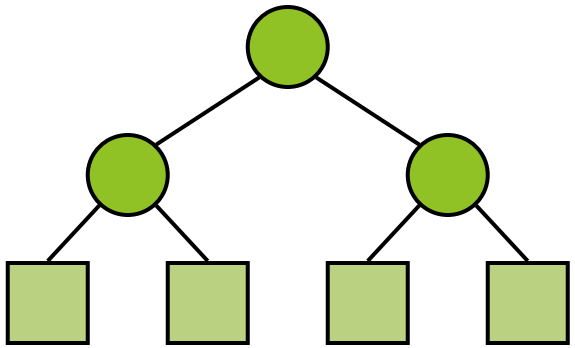
- Example: dining decision

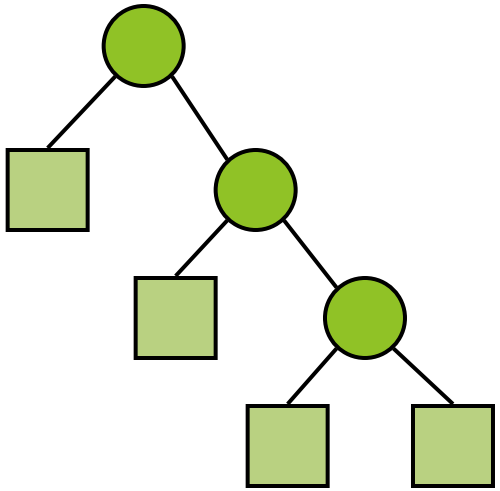


Properties of Binary Trees

- Notation
  - n number of nodes
  - e number of external nodes
  - i number of internal nodes
  - h height

- Properties:
  - $e = i + 1$
  - $n = e + i$
  - $h+1 \le e \le 2^h$
  - $h \le i \le 2^h-1$
  - $2h+1 \le n \le 2^{h+1}-1$
  - $h \le (n - 1)/2$
  - $h \ge \log_2 (n + 1) - 1$





Two extreme cases

Tree Traversal: Pre-order and Post-order

Similar to the methods for general trees

(Except that only two children are considered)

Algorithm B-*preOrder*(*v*)

visit(*v*)

if *v is an internal node* **then**

B-preOrder (*T*,*T.leftChild*(*v*))

B-preOrder (*T*,*T.rightChild*(*v*))

Algorithm B-*postOrder*(*v*)

if *v is an internal node* **then**

B-postOrder (*T*,*T.leftChild*(*v*))

B-postOrder (*T*,*T.rightChild*(*v*))

visit(*v*)

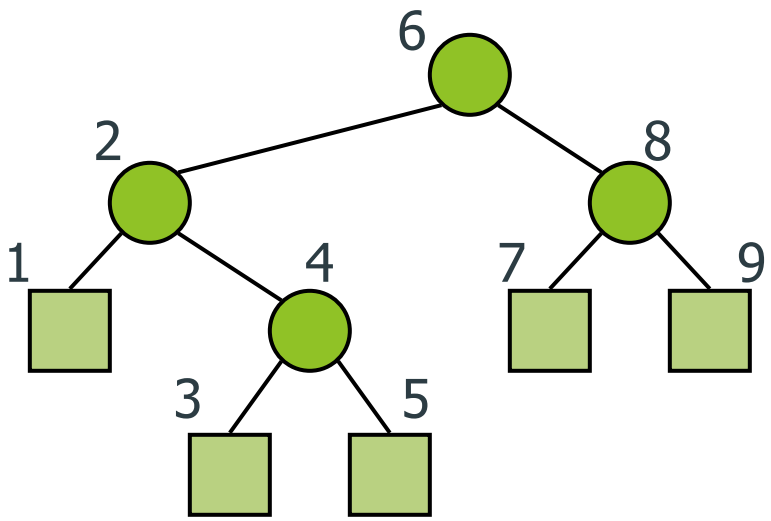
Tree Traversal: Inorder

- In an inorder traversal a node is visited after its left subtree and before its right subtree

- Application:

  Print arithmetic expressions

```
Algorithm inOrder(v)
    if isInternal (v)
        inOrder (leftChild (v))
    visit(v)
    if isInternal (v)
        inOrder (rightChild (v))
```

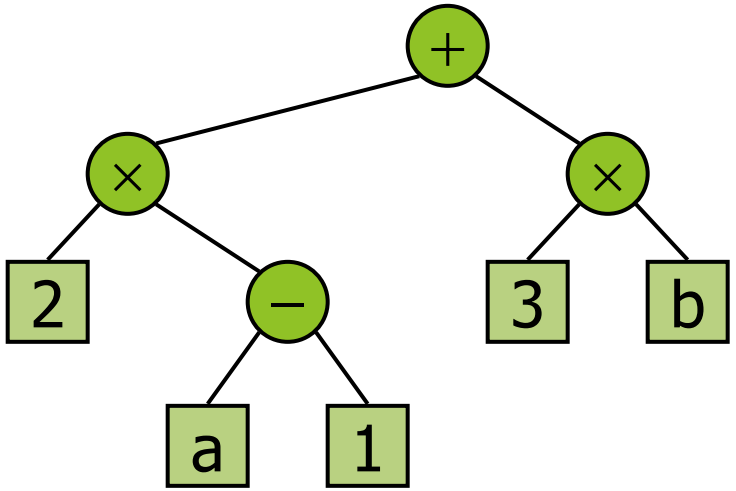


Applies to binary trees only !

Example: Print Arithmetic Expressions

- Specialization of an inorder traversal
  - print operand or operator when visiting node
  - print "(" before traversing left subtree
  - print ")" after traversing right subtree

```
Algorithm printExpr(T, v)
    if isExternal (v)
        print v.element ()
    else
        print "("
        printExpr(T, T.leftChild (v))
        print v.element ()
        printExpr(T, T.rightChild (v))
        print ")"
```

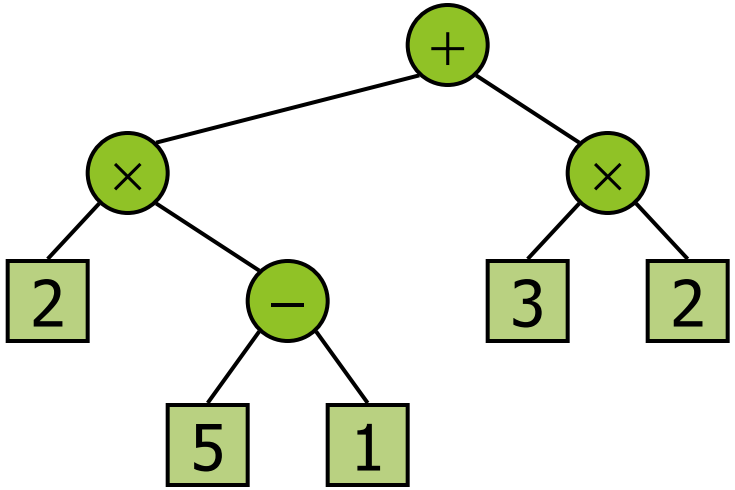


$$((2 \times (a - 1)) + (3 \times b))$$

Example: Evaluate Arithmetic Expressions

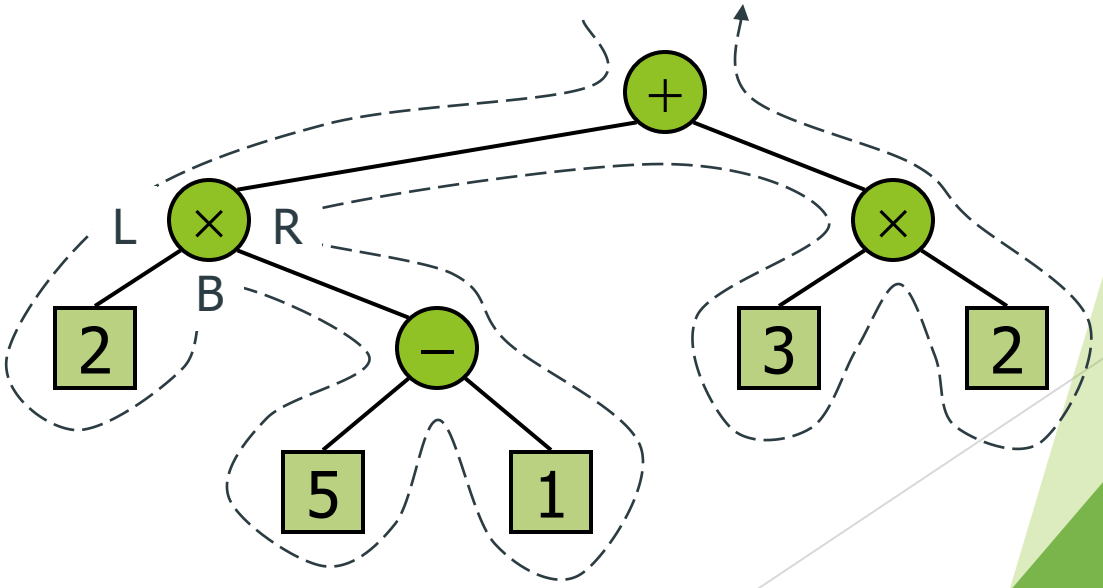
- Specialization of a postorder traversal
  - recursive method returning the value of a subtree
  - when visiting an internal node, combine the values of the subtrees

```
Algorithm evalExpr(v)
    if isExternal (v)
        return v.element ()
    else
        x ← evalExpr(leftChild (v))
        y ← evalExpr(rightChild (v))
        ◊ ← operator stored at v
        return x ◊ y
```



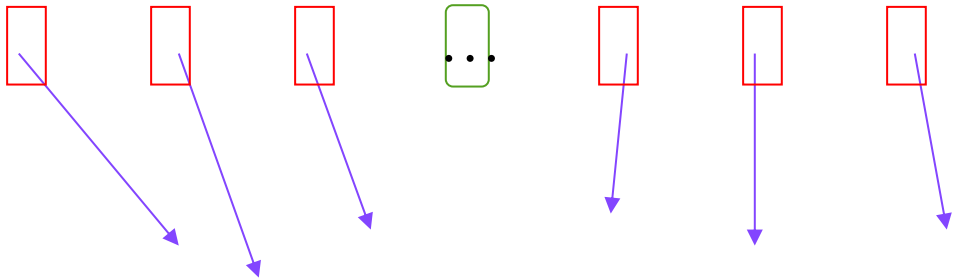
Euler Tour Traversal

- Generic traversal of a **binary** tree (a closed contour enclosing the tree)
- Each node is visited three times: L, R ,B. For an external node, all three happen at the same time
- Special cases: the preorder, postorder and inorder traversals
- Walk around the tree:
  - Pick up “L” (preorder)
  - Pick up “B” (inorder)
  - Pick up “R” (postorder)

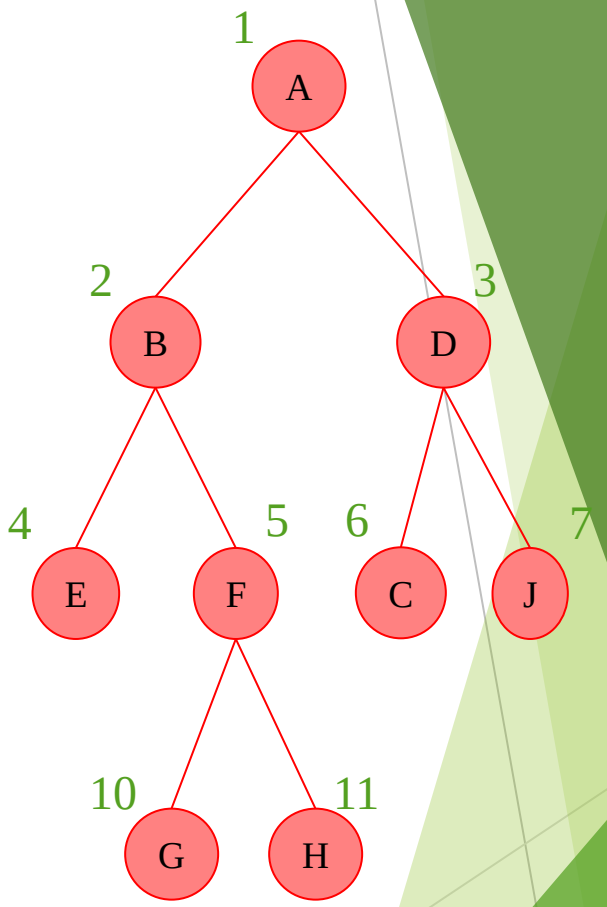


Array-Based Representation of Binary Trees

- nodes are stored in an array

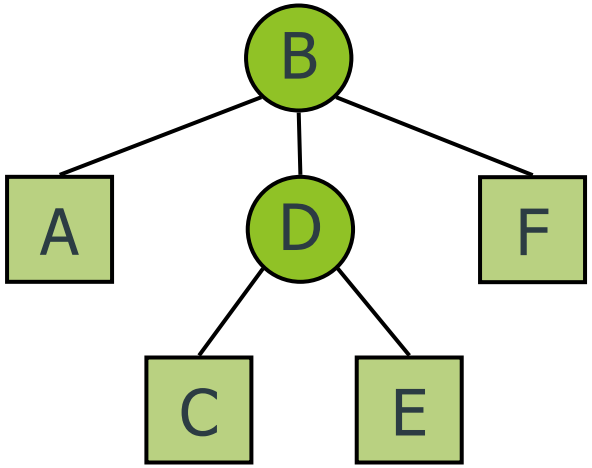


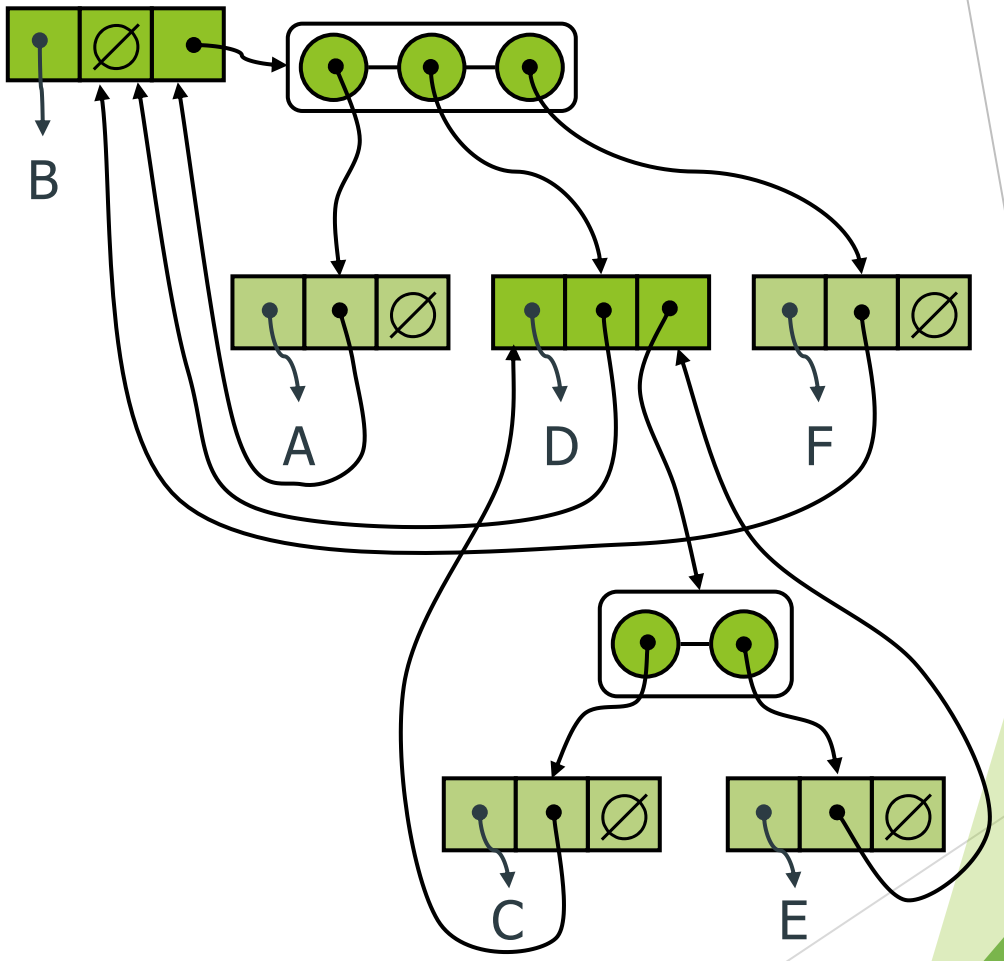
- **let rank(node) be defined as follows:**
  - rank(root) = 1
  - if node is the left child of parent(node), rank(node) = 2*rank(parent(node))
  - if node is the right child of parent(node), rank(node) = 2*rank(parent(node))+1

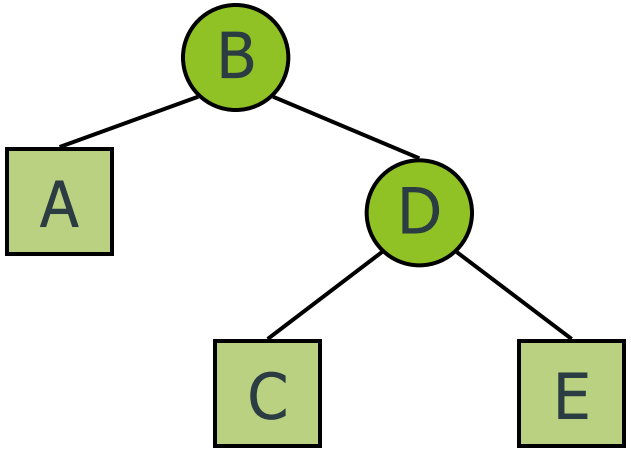
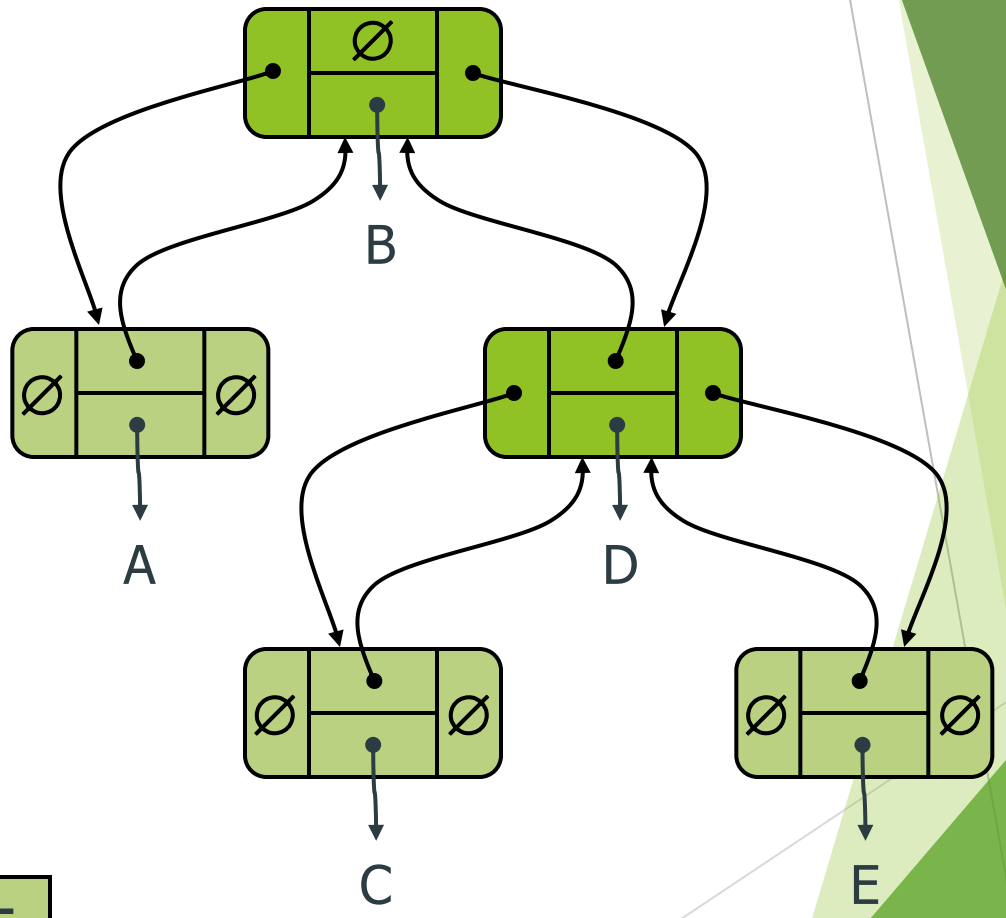


List-Based Representation for Trees

- A node is represented by an object storing
  - Element
  - Parent node
  - Sequence of children nodes
- Node objects implement the Position ADT





List-Based Representation for Binary Trees

- A node is represented by an object storing
  - Element
  - Parent node
  - Left child node
  - Right child node
- Node objects implement the Position ADT